Restaurant Menu Verification System

Comprehensive Project Report

Version 1.0 • March 2026

AI-Assisted Menu Comparison & QA Platform

Prepared by: Development Team

Date: 15 March 2026

Table of Contents

1. Executive Summary
2. Project Overview
3. System Architecture
4. Module Details
5. Application Flow
6. Extraction Pipeline
7. Comparison Engine
8. Technology Stack
9. Current Features
10. Known Limitations
11. Future Roadmap
12. File Inventory

1. Executive Summary

The **Restaurant Menu Verification System** automates menu quality assurance by extracting menus from multiple formats (JSON, PDF, images, DOCX, live websites), normalising data into a shared schema, performing fuzzy matching, and detecting **missing items, extra items, price mismatches, description differences, spelling errors, missing images, and category mismatches**. A Streamlit web interface displays results and offers downloadable PDF reports.

2. Project Overview & Objectives

- Automate menu QA across restaurants and ordering platforms.
- Support diverse inputs: JSON APIs, PDFs, scanned images (OCR), DOCX, websites.
- Detect all categories of menu discrepancies in a single comparison run.
- Provide clear, downloadable PDF reports for restaurant managers and QA teams.
- Handle bot-protected websites via multi-strategy scraping (requests, cloudscraper, cache proxies).

3. System Architecture

The system follows a **layered pipeline architecture**:

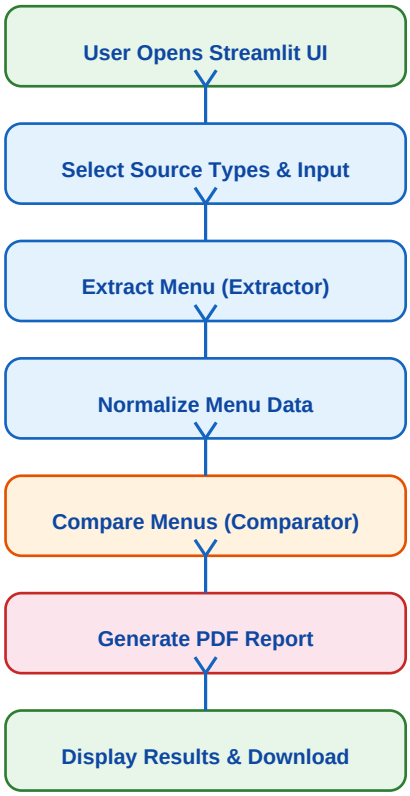
Layer	Package	Responsibility
Presentation	app.py (Streamlit)	UI, user input collection, result display, PDF download
Extraction	extractor/	Read menus from JSON, PDF, Image, DOCX, Website sources
Processing	processing/	Normalize and clean all menu data into a unified schema
Comparison	comparator/	Fuzzy-match items and detect all mismatch types
Reporting	reports/	Build downloadable PDF verification reports

4. Module Details

Module	File	Key Functions / Classes
Extractor Factory	extractor/factory.py	extract_menu_from_source() - routes to correct reader
JSON Reader	extractor/json_reader.py	read_json_menu() - parse JSON text or file bytes
PDF Reader	extractor/pdf_reader.py	read_pdf_menu() - extract text via pdfplumber
Image OCR	extractor/image_ocr.py	read_image_menu() - Tesseract OCR on uploaded images
DOCX Reader	extractor/docx_reader.py	read_docx_menu() - extract from Word documents
Web Scraper	extractor/web_scraper.py	read_website_menu() - LD+JSON, HTML tables, text fallback
Normalizer	processing/normalize_menu.py	normalize_menu(), parse_menu_text(), coerce_price()
Item Matcher	comparator/item_matcher.py	best_item_match(), similarity() - fuzzy matching
Price Checker	comparator/price_checker.py	price_difference() - detect price mismatches
Desc Checker	comparator/description_checker.py	descriptions_differ(), detect_spelling_errors()
Image Checker	comparator/image_checker.py	missing_image_issue() - flag missing images
Menu Comparator	comparator/menu_comparator.py	compare_menus() - orchestrates all checks
PDF Report	reports/pdf_report.py	build_pdf_bytes() - generate downloadable PDF

5. Application Flow (Flowchart)

Main application pipeline from user interaction to report download:

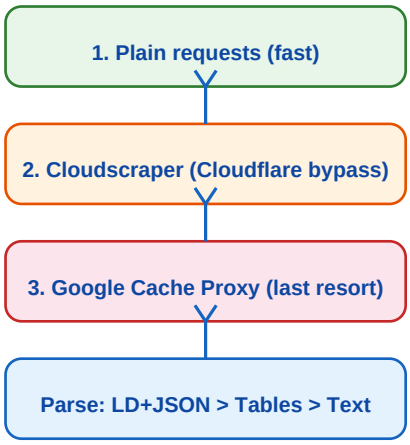


6. Extraction Pipeline Flow

The Extractor Factory routes input to the appropriate reader:

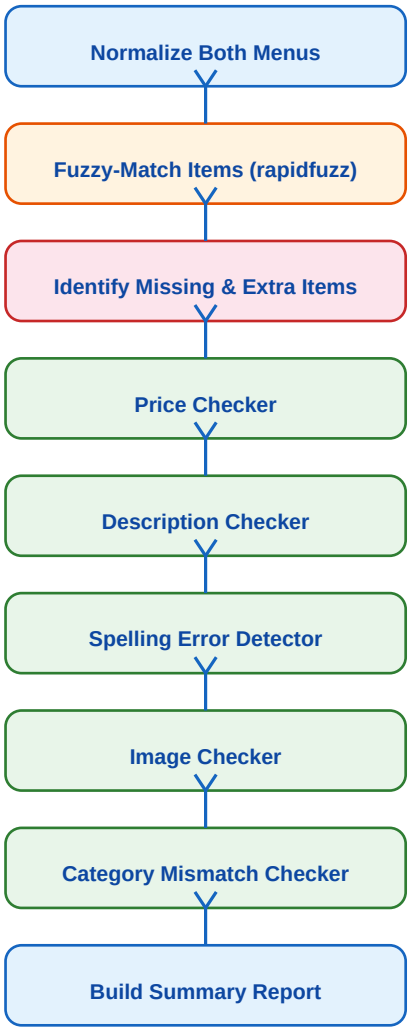
Source Type	Reader Module	Strategy
JSON Text / File	json_reader.py	json.loads() on text or decoded file bytes
PDF Upload	pdf_reader.py	pdfplumber page-by-page text extraction
Image Upload	image_ocr.py	Tesseract OCR via pytesseract
Word Document	docx_reader.py	python-docx paragraph + table extraction
Website URL	web_scraper.py	LD+JSON > HTML tables > plain text (3 strategies)

Web Scraper Multi-Strategy Approach:



7. Comparison Engine Flow

The comparator aligns items via fuzzy matching, then runs all check modules:



Mismatch Types Detected:

Check	Module	Logic
Missing Items	item_matcher	Reference items not found in our menu (score < 84)
Extra Items	item_matcher	Our items not matched to any reference item
Price Mismatch	price_checker	Absolute difference > $0.01 tolerance
Description Diff	description_checker	SequenceMatcher ratio < 0.88 or word count gap
Spelling Errors	description_checker	Token-level fuzzy match ratio 0.82-0.99
Missing Images	image_checker	Our item has no image URL when reference does
Category Mismatch	menu_comparator	Category name differs between matched items

8. Technology Stack

Technology	Purpose	Version / Notes
Python 3.10+	Core language	3.10 - 3.13 tested
Streamlit	Web UI framework	Interactive dashboard with widgets
reportlab	PDF generation	v4.4 - vector PDF reports
rapidfuzz	Fuzzy string matching	Token sort ratio for item alignment
pdfplumber	PDF text extraction	Page-by-page text extraction
pytesseract	OCR for images	Requires Tesseract engine on PATH
python-docx	Word document parsing	Paragraph + table extraction
BeautifulSoup4	HTML parsing	LD+JSON, tables, text from websites
requests	HTTP client	Primary web fetching strategy
cloudscraper	Anti-bot bypass	Cloudflare JS challenge handling
Pillow	Image processing	Image loading for OCR pipeline

9. Current Features

- **Multi-source extraction:** JSON, PDF, Image (OCR), DOCX, Website URL
- **Smart web scraping:** 3-strategy fallback (requests > cloudscraper > cache proxy)
- **LD+JSON extraction:** Parses structured restaurant data from Schema.org markup
- **Unified normalization:** All sources normalized to {category, item, price, description, image}
- **Fuzzy item matching:** rapidfuzz token_sort_ratio with 84% threshold + category bonus
- **7 mismatch types:** Missing, Extra, Price, Description, Spelling, Image, Category
- **Interactive UI:** Streamlit with tabs, metrics, expanders, and dataframe views
- **PDF report download:** Professional verification report via reportlab
- **JSON report export:** Full structured report displayed in-app
- **Unit test suite:** Tests for comparator logic and web scraper

10. Known Limitations

- Bot-protected sites may still block scraping (Cloudflare Enterprise, reCAPTCHA)
- OCR accuracy depends on image quality and Tesseract configuration
- Fuzzy matching threshold (84%) may need tuning for different cuisines
- No persistent storage - results are session-only
- No user authentication or multi-tenant support
- PDF report is basic text-only (no charts or visual analytics)
- No CI/CD pipeline or automated deployment configured

11. Future Roadmap & Next Steps

Phase 1 - Short Term (Next 2-4 weeks):

- Add Selenium/Playwright browser automation for JavaScript-rendered restaurant sites
- Integrate OpenAI/Gemini API for intelligent menu item categorization
- Add batch comparison mode - compare multiple restaurant locations at once
- Enhance PDF report with charts, color-coded severity, and visual dashboards
- Add CSV/Excel export alongside PDF

Phase 2 - Medium Term (1-2 months):

- Build a database layer (PostgreSQL/SQLite) for historical comparison tracking
- Add scheduled automated comparisons with email/Slack notifications
- Create a REST API endpoint for programmatic access (FastAPI)
- Implement role-based access control and user management
- Add image comparison using perceptual hashing (imagehash)
- Support menu translations and multi-language comparison

Phase 3 - Long Term (3-6 months):

- ML-based menu categorization model trained on restaurant menu datasets
- Real-time menu monitoring dashboard with change detection alerts
- Integration with POS systems (Square, Toast, Clover) for live menu sync
- Mobile-friendly PWA version of the verification interface
- Containerized deployment (Docker) with CI/CD pipeline (GitHub Actions)
- Multi-restaurant chain management with centralized reporting
- Compliance checking against food labeling regulations

12. File Inventory

File	Lines	Purpose
app.py	124	Streamlit UI - main entry point
extractor/__init__.py	2	Package init, re-exports factory
extractor/factory.py	23	Routes source type to correct reader
extractor/json_reader.py	11	JSON text/file parsing
extractor/pdf_reader.py	15	PDF text extraction via pdfplumber
extractor/image_ocr.py	16	Image OCR via pytesseract
extractor/docx_reader.py	20	Word document extraction
extractor/web_scraper.py	~160	Multi-strategy website menu scraping
processing/normalize_menu.py	117	Menu normalization and text parsing
comparator/menu_comparator.py	85	Orchestrates all comparison checks
comparator/item_matcher.py	45	Fuzzy item matching with rapidfuzz
comparator/price_checker.py	14	Price difference detection
comparator/description_checker.py	47	Description diff and spelling checks
comparator/image_checker.py	8	Missing image detection
reports/pdf_report.py	60	PDF report builder with reportlab
tests/test_menu_comparator.py	69	Unit tests for comparison engine
tests/test_web_scraper.py	~50	Unit tests for web scraper

--- End of Report ---